Name: sara gamal Group: SE3

Lab3

1- Start a resource-limited container

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task>docker run -d --name limited-container --memory=200m --cpus=0.5 nginx
f675d4898f6c0bd0207749b5fd890b9e2ee646b26e76e528d78d14cd4a5a5542

C:\docker-task>docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS          PORTS                    NAMES
f675d4898f6c   nginx             "/docker-entrypoint.…"   23 seconds ago   Up 22 seconds   80/tcp                   limited-container
f7cd1b271b19   archproj-server2  "python server.py"       15 months ago    Up 3 minutes    0.0.0.0:5002->5000/tcp   archproj-server2-1
30fdb010ae8a   archproj-server1  "python server.py"       15 months ago    Up 3 minutes    0.0.0.0:5001->5000/tcp   archproj-server1-1

C:\docker-task>
```

2- Display the container's status before and after compression

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task>docker stats limited-container
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
f675d4898f6c   limited-container  0.00%     0B / 0B             0.00%     0B / 0B   0B / 0B     0
CONTAINER ID   NAME               CPU %     MEM USAGE / LIMIT   MEM %     NET I/O   BLOCK I/O   PIDS
```

```
C:\docker-task>docker exec -it limited-container bash
root@f675d4898f6c:/#
```

```
C:\docker-task>docker exec -it limited-container bash
root@f675d4898f6c:/# apt update
apt install -y stress
stress --cpu 2 --vm 1 --vm-bytes 128M
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [111 kB]
Fetched 202 kB in 0s (410 kB/s)
All packages are up to date.
stress is already the newest version (1.0.7-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
stress: info: [68] dispatching hogs: 2 cpu, 0 io, 1 vm, 0 hdd
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task>docker stats limited-container
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   4.76%     63.64MiB / 200MiB   31.82%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   4.76%     63.64MiB / 200MiB   31.82%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.32%    45.25MiB / 200MiB   22.63%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.32%    45.25MiB / 200MiB   22.63%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.44%    155.8MiB / 200MiB   77.89%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.44%    155.8MiB / 200MiB   77.89%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.40%    125.8MiB / 200MiB   62.90%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.40%    125.8MiB / 200MiB   62.90%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.71%    131.8MiB / 200MiB   65.91%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.71%    131.8MiB / 200MiB   65.91%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.24%    47.6MiB / 200MiB    23.80%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.24%    47.6MiB / 200MiB    23.80%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.02%    119.8MiB / 200MiB   59.88%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.02%    119.8MiB / 200MiB   59.88%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.22%    107.8MiB / 200MiB   53.90%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   50.22%    107.8MiB / 200MiB   53.90%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.86%    47.61MiB / 200MiB   23.80%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
f675d4898f6c   limited-container   49.86%    47.61MiB / 200MiB   23.80%    216kB / 5.87kB  0B / 0B     26
CONTAINER ID   NAME                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O   PIDS
```

3-Steps of Building Basic Docker Image

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task>mkdir basic-docker

C:\docker-task>cd basic-docker

C:\docker-task\basic-docker>
```

```
FROM nginx
COPY index.html /usr/share/nginx/html/index.html
```

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\basic-docker>docker build -t my-nginx-image .
[+] Building 2.0s (7/7) FINISHED                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                           0.0s
 => => transferring dockerfile: 97B                                                            0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                0.0s
 => [internal] load .dockerignore                                                              0.0s
 => => transferring context: 2B                                                                0.0s
 => [internal] load build context                                                              0.0s
 => => transferring context: 58B                                                               0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3  1.6s
 => => resolve docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3  1.5s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                                     0.0s
 => exporting to image                                                                         0.2s
 => => exporting layers                                                                        0.1s
 => => exporting manifest sha256:e59da1f5bdfd75d089290d39c14e5747276d4447dc58d928bee7f976884967bf  0.0s
 => => exporting config sha256:7d65992ccf08da0368b087f9bf2e07e9d52fbcf8686d4315f08f4d3780fa8c73  0.0s
 => => exporting attestation manifest sha256:afb67ffb4147da391414b7cc4bef24c7210af88a10bfc6e293b1116cd0e42235  0.0s
 => => exporting manifest list sha256:34693b787e2c69e52ec3ed85c2cdb1c28906346abc3c40d3b55fc7b764fcf9bc  0.0s
 => => naming to docker.io/library/my-nginx-image:latest                                       0.0s
 => => unpacking to docker.io/library/my-nginx-image:latest                                    0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/n2nzqd3n9z3fgne0x6uzgl5sl

C:\docker-task\basic-docker>
```

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\basic-docker>docker run -d -p 8081:80 my-nginx-image
744854c73e89889ddf717dfc30935f3fe44d93de573315e039c8d42d0bc1b343

C:\docker-task\basic-docker>docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                    NAMES
744854c73e89   my-nginx-image     "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes   0.0.0.0:8081->80/tcp     sharp_burnell
f675d4898f6c   nginx              "/docker-entrypoint.…"   3 hours ago     Up 3 hours     80/tcp                   limited-container
f7cd1b271b19   archproj-server2   "python server.py"       15 months ago   Up 3 hours     0.0.0.0:5002->5000/tcp   archproj-server2-1
30fdb010ae8a   archproj-server1   "python server.py"       15 months ago   Up 3 hours     0.0.0.0:5001->5000/tcp   archproj-server1-1

C:\docker-task\basic-docker>
```

localhost:8081

Hello Docker

4-Steps of Multi-Stage Building of Docker Image

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\multi-stage>docker build -t multi-stage-app .
[+] Building 5.8s (9/9) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 247B                                                                                0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                                                     0.0s
 => [internal] load metadata for docker.io/library/busybox:latest                                                   3.7s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 2B                                                                                     0.0s
 => [builder 1/2] FROM docker.io/library/busybox:latest@sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4207dd76f  1.5s
 => => resolve docker.io/library/busybox:latest@sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4207dd76f  0.0s
 => => sha256:61dfb50712f5ff92c880813210257a42169ff0937896ae95dab763582cc380e2 2.21MB / 2.21MB                     1.4s
 => => extracting sha256:61dfb50712f5ff92c880813210257a42169ff0937896ae95dab763582cc380e2                           0.1s
 => CACHED [stage-1 1/2] FROM docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e  0.0s
 => => resolve docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e  0.0s
 => [builder 2/2] RUN echo "Hello Multi-Stage" > /tmp/index.html                                                    0.2s
 => [stage-1 2/2] COPY --from=builder /tmp/index.html /usr/share/nginx/html/index.html                              0.0s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:6cc9f70633a6b2e7f24b68825531ff0b2043b7e46493bc75a7e60bcf243a24ef                   0.0s
 => => exporting config sha256:1d61f0df1f72adedcfedef0c1c81820958d0cab7fb3c76b69562a4387a2ee178                     0.0s
 => => exporting attestation manifest sha256:9d92b1aef73a6ad1f505b1290b5289baeba0fb33c718e263eb4cfc9aefa66047       0.0s
 => => exporting manifest list sha256:b054152e180e3f9a65b8859e1a7afc24d2df175d11262d82a6b43dd661b88da5              0.0s
 => => naming to docker.io/library/multi-stage-app:latest                                                           0.0s
 => => unpacking to docker.io/library/multi-stage-app:latest                                                        0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/kwhdds4836zw6o3d8fj8najjn

C:\docker-task\multi-stage>
```

5-Create Local Cluster using ""kind

```
C:\Windows\System32\cmd.e

C:\docker-task\multi-stage>curl -Lo kind.exe https://kind.sigs.k8s.io/dl/v0.23.0/kind-windows-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    99  100    99    0     0    218      0 --:--:-- --:--:-- --:--:--   220
  0     0    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--     0
100 6547k  100 6547k    0     0  1805k      0  0:00:03  0:00:03 --:--:-- 3309k

C:\docker-task\multi-stage>dir
 Volume in drive C has no label.
 Volume Serial Number is DC6E-94D5

 Directory of C:\docker-task\multi-stage

03/11/2026  10:23 PM    <DIR>          .
03/11/2026  08:51 PM    <DIR>          ..
03/11/2026  09:15 PM               208 Dockerfile
03/11/2026  10:23 PM         6,704,128 kind.exe
               2 File(s)      6,704,336 bytes
               2 Dir(s)   7,113,154,560 bytes free

C:\docker-task\multi-stage>
```

```
C:\docker-task\multi-stage>.\kind.exe version
kind v0.23.0 go1.21.10 windows/amd64

C:\docker-task\multi-stage>.\kind.exe create cluster --name lab-cluster
Creating cluster "lab-cluster" ...
 • Ensuring node image (kindest/node:v1.30.0) 🖼 ...
```

```
C:\docker-task\multi-stage>kubectl config get-contexts
CURRENT   NAME               CLUSTER            AUTHINFO           NAMESPACE
*         kind-lab-cluster   kind-lab-cluster   kind-lab-cluster

C:\docker-task\multi-stage>kubectl config use-context kind-lab-cluster
Switched to context "kind-lab-cluster".

C:\docker-task\multi-stage>kubectl get nodes
NAME                       STATUS   ROLES           AGE   VERSION
lab-cluster-control-plane  Ready    control-plane   54s   v1.30.0

C:\docker-task\multi-stage>
```

6-Copy images into the cluster

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\multi-stage>kind load docker-image my-nginx-image --name lab-cluster
Image: "my-nginx-image" with ID "sha256:34693b787e2c69e52ec3ed85c2cdb1c28906346abc3c40d3b55fc7b764fcf9bc" not yet present on node "lab-cluster-control-plane
", loading...

C:\docker-task\multi-stage>
```

7-Get the deployment description

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\multi-stage>kind load docker-image my-nginx-image --name lab-cluster
Image: "my-nginx-image" with ID "sha256:34693b787e2c69e52ec3ed85c2cdb1c28906346abc3c40d3b55fc7b764fcf9bc" not yet present on node "lab-cluster-control-plane
", loading...

C:\docker-task\multi-stage>kubectl create deployment nginx-deployment --image=my-nginx-image
deployment.apps/nginx-deployment created

C:\docker-task\multi-stage>
```

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\docker-task\multi-stage>kind load docker-image my-nginx-image --name lab-cluster
Image: "my-nginx-image" with ID "sha256:34693b787e2c69e52ec3ed85c2cdb1c28906346abc3c40d3b55fc7b764fcf9bc" not yet present on node "lab-cluster-control-plane
", loading...

C:\docker-task\multi-stage>kubectl create deployment nginx-deployment --image=my-nginx-image
deployment.apps/nginx-deployment created

C:\docker-task\multi-stage>kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   0/1     1            0           31s

C:\docker-task\multi-stage>
```

```
C:\docker-task\multi-stage>kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   0/1     1            0           31s

C:\docker-task\multi-stage>kubectl describe deployment nginx-deployment
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Fri, 13 Mar 2026 17:06:42 +0200
Labels:                 app=nginx-deployment
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=nginx-deployment
Replicas:               1 desired | 1 updated | 1 total | 0 available | 1 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-deployment
  Containers:
   my-nginx-image:
    Image:         my-nginx-image
    Port:          <none>
    Host Port:     <none>
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      False   MinimumReplicasUnavailable
  Progressing    True    ReplicaSetUpdated
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-5b479f6666 (1/1 replicas created)
Events:
  Type    Reason             Age    From                   Message
  ----    ------             ----   ----                   -------
  Normal  ScalingReplicaSet  2m29s  deployment-controller  Scaled up replica set nginx-deployment-5b479f6666 to 1

C:\docker-task\multi-stage>
```

1. Why do namespaces alone not guarantee fair resource use?

Namespaces provide **isolation** (separate processes, network, filesystem), but they do not limit how much CPU or memory a container can use.
A container can still consume excessive resources and affect others.
Therefore, namespaces alone cannot ensure fair resource distribution.

2. How do cgroups improve cluster stability?

cgroups enforce **resource limits** (CPU, memory, etc.) on containers.
This prevents any single container from exhausting system resources.
As a result, the system remains stable and all containers get a fair share.

3. Why is Docker image layering important for large-scale orchestration?

Docker images are built in **layers**, which allows:

- Reusing unchanged layers
- Faster builds and deployments
- Reduced storage and network usage

This is crucial at scale because many containers can share the same layers efficiently.

4. What does Kubernetes mean by desired state?

The **desired state** is the state you define in configuration files (e.g., number of replicas).
Kubernetes continuously works to ensure the system matches this state.

For example, if you specify 3 Pods, Kubernetes will always try to keep 3 running.

5. How is self-healing different from traditional manual operations?

In traditional systems, failures are handled **manually** (restart services, fix issues).
In Kubernetes, self-healing is **automatic**:

- Failed containers are restarted
- Broken Pods are replaced
- Unresponsive services are recovered

This reduces downtime and removes the need for manual intervention.

6. Why are readiness probes and liveness probes not interchangeable?

- **Readiness probe**: checks if the container is ready to serve traffic
- **Liveness probe**: checks if the container is still alive

If readiness fails → traffic is stopped
If liveness fails → container is restarted

They serve different purposes and cannot replace each other.

7. What is one limitation of observing scheduling in a single-node kind cluster compared with a real multi-node cluster?

In a single-node cluster, all Pods are scheduled on the **same node**, so you cannot observe real scheduling decisions like:

- Load balancing across nodes
- Node selection strategies
- Failure handling between nodes

This makes it less realistic compared to a multi-node cluster.